

Vectorization of Neural Networks (Optional)

The ability to **vectorize** neural networks is a primary reason they have scaled so successfully. This approach allows them to leverage parallel computing hardware, like **GPUs**, which are highly optimized for a specific mathematical operation: **matrix multiplication**.

Non-Vectorized vs. Vectorized Implementation

This section contrasts the two ways to implement a forward propagation step for a single layer.

- **Non-Vectorized (Loop-based):**
 - This is the "from-scratch" method seen previously.
 - It uses a **for loop** to iterate through each neuron (j) in the layer.
 - It computes the z and a for each neuron **one at a time (sequentially)**.
 - This method is easy to understand but very **slow**.
 - **Vectorized (Matrix-based):**
 - This method replaces the entire `for` loop with a single operation.
 - It performs all the neuron computations **at the same time (in parallel)** using matrix multiplication.
 - This is exceptionally **fast** on modern hardware.
-

The Vectorized Implementation

The code for a dense layer is radically simplified.

1. Compute Z :

- $Z = \text{np.matmul}(A_{\text{in}}, W) + B$

2. Compute A_{out} :

- $A_{\text{out}} = g(Z)$ (where g is the sigmoid function, applied to every element of the Z matrix).
-

Key Concepts in This Implementation

- **`np.matmul`:** This is the NumPy function for **matrix multiplication**. It is the core of the vectorized implementation, replacing the explicit `for` loop.
- **Key Data Representation Change:**
 - To make matrix multiplication work, **all variables** are now treated as **2D arrays (matrices)**, not 1D vectors.

- **A_in** (the input activations, or x) is now a 2D array (e.g., a 1x2 matrix: $\begin{bmatrix} \dots \end{bmatrix}$).
- **w** (the weights) is a 2D array (matrix).
- **B** (the biases) is also a 2D array (e.g., a 1x3 matrix: $\begin{bmatrix} \dots \end{bmatrix}$).
- The outputs **Z** and **A_out** will also be 2D arrays (matrices).

Matrix Multiplication (Optional)

This section explains how matrix multiplication works, building up from the simpler vector dot product. This is the core mathematical operation that makes vectorization possible.

1. Vector-Vector Dot Product

The **dot product** is a fundamental operation where you take two vectors of the same size, multiply their corresponding elements, and sum the results.

- **Example:**
 - $\vec{a} = [1, 2]$
 - $\vec{w} = [3, 4]$
 - **Dot Product:** $z = (1 \times 3) + (2 \times 4) = 3 + 8 = 11$
 - **General Formula:** $z = a_1w_1 + a_2w_2 + \dots + a_nw_n$
-

2. The Transpose and the "Row $\times$ Column" Rule

- A **column vector** is a matrix with one column (e.g., a 2x1 matrix).
 - A **row vector** is a matrix with one row (e.g., a 1x2 matrix).
 - The **transpose** operation (denoted by A^T) converts a column vector into a row vector (it lays the vector "on its side").
 - **Key Equivalence:** The vector dot product $\vec{a} \cdot \vec{w}$ is mathematically equivalent to the matrix multiplication of $\vec{a}^T$ (a row vector) and $\vec{w}$ (a column vector).
 - $z = \vec{a} \cdot \vec{w} \equiv z = \vec{a}^T \times \vec{w}$
-

3. Vector-Matrix Multiplication

This is the process of multiplying a row vector by a matrix.

- **Rule:** You perform a dot product between the **row vector** and **each column** of the matrix. The results form a new row vector.
- **Example:**
 - $\vec{a}^T = [1, 2]$ (a 1x2 matrix)
 - $W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix}$ (a 2x2 matrix, with columns $\begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and $\begin{bmatrix} 5 \\ 6 \end{bmatrix}$)
- **Calculation:**
 1. **First element of output:** $(\vec{a}^T) \cdot (\text{Col 1 of } W)$

- $[1, 2] \cdot [3, 4] = (1 \times 3) + (2 \times 4) = 11$

2. **Second element of output:** $(\vec{a}^T) \cdot (\text{Col 2 of } W)$

- $[1, 2] \cdot [5, 6] = (1 \times 5) + (2 \times 6) = 17$

- **Result:** $Z = [11, 17]$ (a 1x2 matrix)

4. Matrix-Matrix Multiplication

This is the general form. It simply repeats the vector-matrix multiplication process for **every row** of the first matrix.

- **Rule:** The element at (row i , col j) of the output matrix is the dot product of (row i) of the *first* matrix and (col j) of the *second* matrix.

- **Example:** $Z = A^T \times W$

- $A^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix}$

- $W = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix}$

- **Calculation:**

1. **Row 1 of Z:** (Row 1 of A^T) $\times$ W

- $([1, 2] \cdot [3, 4]) = 11$

- $([1, 2] \cdot [5, 6]) = 17$

- $\rightarrow$ First row of Z is $[11, 17]$

2. **Row 2 of Z:** (Row 2 of A^T) $\times$ W

- $([-1, -2] \cdot [3, 4]) = (-3) + (-8) = -11$

- $([-1, -2] \cdot [5, 6]) = (-5) + (-12) = -17$

- $\rightarrow$ Second row of Z is $[-11, -17]$

- **Final Result:** $Z = \begin{bmatrix} 11 & 17 \\ -11 & -17 \end{bmatrix}$

General Matrix Multiplication (Optional)

This section provides the general rule for matrix multiplication, which is the core operation for vectorizing a neural network.

The Transpose Operation (A^T)

- A **matrix** is a 2D array of numbers, defined by its (`rows`, `columns`).
 - The **transpose** of a matrix, A^T , is found by taking the **columns** of A and turning them into the **rows** of A^T .
 - **Example:** If A is a **2x3** matrix (2 rows, 3 cols), then A^T will be a **3x2** matrix (3 rows, 2 cols).
-

The Rule for Matrix Multiplication: $Z = A \times W$

The core rule for multiplying two matrices (e.g., A^T and W) is to compute the dot product of rows from the first matrix with columns from the second.

- To find the element in **row i** and **column j** of the output matrix Z , you take the dot product of **row i of the first matrix** and **column j of the second matrix**.
 - $Z(i, j) = (\text{Row } i \text{ of } A^T) \cdot (\text{Column } j \text{ of } W)$
-

Example Calculation

Let's calculate a few elements of $Z = A^T \times W$.

- **To find $Z(1, 1)$ (Row 1, Col 1):**
 - (Row 1 of A^T) $\cdot$ (Col 1 of W)
 - $[1, 2] \cdot [3, 4] = (1 \times 3) + (2 \times 4) = \mathbf{11}$
- **To find $Z(3, 2)$ (Row 3, Col 2):**
 - (Row 3 of A^T) $\cdot$ (Col 2 of W)
 - $[0.1, 0.2] \cdot [5, 6] = (0.1 \times 5) + (0.2 \times 6) = 0.5 + 1.2 = \mathbf{1.7}$
- **To find $Z(2, 3)$ (Row 2, Col 3):**
 - (Row 2 of A^T) $\cdot$ (Col 3 of W)
 - $[-1, -2] \cdot [7, 8] = (-1 \times 7) + (-2 \times 8) = -7 - 16 = \mathbf{-23}$

This process is repeated for every element to compute the full output matrix Z .

Dimension Rules for Multiplication

You cannot multiply matrices of any arbitrary size. They must follow two specific rules.

- **1. Requirement:** To multiply $\text{Matrix1} \times \text{Matrix2}$, the number of **columns** in Matrix1 must **equal** the number of **rows** in Matrix2 .
 - Example: A^T (a **3x2** matrix) $\times W$ (a **2x4** matrix).
 - The "inner dimensions" (2 and 2) match, so the operation is valid.
 - This is required because the dot product must be between vectors of the same length.
 - **2. Output Shape:** The resulting matrix, Z , will have the "outer dimensions."
 - Shape: (Number of **rows** from Matrix1) $\times$ (Number of **columns** from Matrix2).
 - Example: Z will be a **3x4** matrix.
-

Key Takeaway

This matrix multiplication rule is the key to **vectorization**. It allows a neural network to perform all the dot products for an entire layer in one highly efficient operation, which is why `np.matmul` runs so much faster than a `for` loop.

Vectorized Implementation in Practice (Optional)

This section connects the theory of matrix multiplication directly to the efficient, vectorized code used to implement a neural network layer.

The Goal: Replacing Loops with Matrix Multiplication

Instead of using a `for` loop to calculate the activation of each neuron one by one, we can use a single matrix multiplication operation to compute the activations for the *entire layer* at once.

Setting Up the Matrices

To use matrix multiplication, all our data (inputs, weights, and biases) must be 2D arrays (matrices).

1. Input Activations (**A_in** or **X**):

- The input features for a single example are represented as a **row vector** (a $1 \times n$ matrix, where n is the number of features).
- `X = np.array([[200, 17]])` (This is a 1×2 matrix)

2. Weights (**W**):

- The weight vectors for all neurons in the *current* layer are stacked together as **columns**.
- If Layer 1 has 3 neurons and 2 input features, `W` will be a **2×3 matrix**.
- `W = np.array([[w1_1, w1_2, w1_3], [w2_1, w2_2, w2_3]])`

3. Biases (**B**):

- The biases for all neurons in the current layer are stored in a **row vector**.
 - `B = np.array([b_1, b_2, b_3])` (This is a 1×3 matrix)
-

The Vectorized Computation

The entire `for` loop from the "from-scratch" implementation is replaced by two lines of code:

1. Compute **Z** (the linear part):

- `Z = np.matmul(X, W) + B`
- This single `np.matmul(X, W)` operation performs the dot product of the input row vector `X` with *each column* of `W`, producing a 1×3 row vector.
- The bias vector `B` is then added to this result.
- The resulting `Z` is a 1×3 matrix containing all the z values for the layer: `[[z_1, z_2,`

`z_3]]`.

2. Compute **A_out** (the activation):

- **A_out** = **g**(Z)
- The sigmoid activation function **g** is applied *element-wise* to the Z matrix.
- The result **A_out** is a 1x3 matrix with the final activations for the layer: `[[a_1, a_2, a_3]]`.

Why This is So Powerful

This vectorized approach is the key to deep learning's performance.

- **Efficiency:** Modern hardware (CPUs and especially GPUs) is highly optimized to perform matrix multiplications (`matmul`) incredibly fast, far faster than a Python `for` loop.
- **Simplicity:** The code becomes much cleaner and more compact.

Note on Syntax: In some Python code, you may also see matrix multiplication written with the `@` operator: `Z = X @ W + B`. This is an alternative syntax for `np.matmul(X, W)`.